

Task on CloudTrail & CloudWatch

1.Enable CloudTrail monitoring and store the events in S3 and CloudWatch log events.

Task 1 — CloudTrail → S3 + CloudWatch Logs

Target:

AWS API activity → CloudTrail → S3 bucket
CloudTrail → CloudWatch Logs

Task 1: Enable CloudTrail and Store Events in S3 and CloudWatch Logs

Objective

To enable AWS CloudTrail for recording AWS account activity and deliver the CloudTrail events to both **Amazon S3** and **Amazon CloudWatch Logs**.

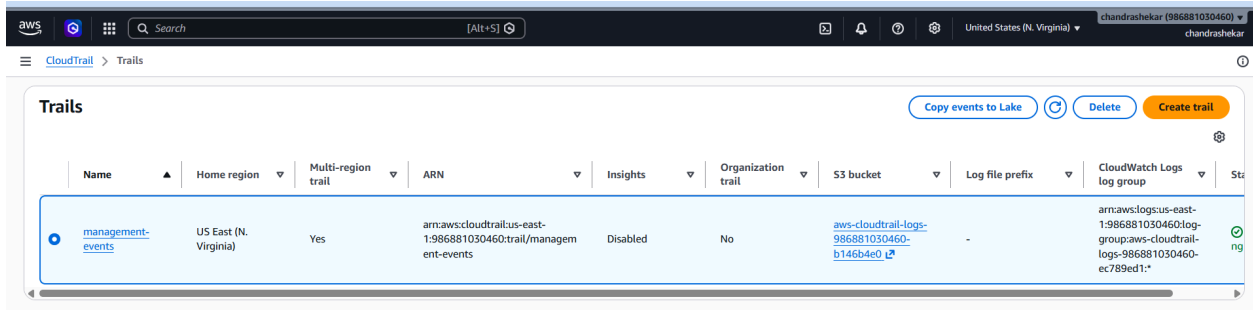
Steps Performed

Step 1 — Open CloudTrail

- Log in to the AWS Management Console.
- Search for **CloudTrail**.
- Navigate to **CloudTrail** → **Trails**.
- An existing trail named **management-events** was available and was used for this task.
- The trail was configured as a **Multi-Region trail**.



Screenshot 1: Use your screenshot showing the **CloudTrail Trails** page with **management-events**.



Step 2 — Verify S3 logging

- Open the **management-events** trail.
- Verify that **Trail logging = Logging**.
- Verify that an **S3 bucket** is configured as the trail log location.
- CloudTrail stores the generated audit log files in this S3 bucket.

 **Screenshot 2:** Use your screenshot showing:

Trail logging: Logging

Trail name: management-events

Trail log location: S3 bucket

This is the screenshot where you can also see **Last log file delivered**.

Step 3 — Configure CloudWatch Logs

Navigate to:

CloudTrail → **management-events** → **Edit** → **CloudWatch Logs**

Configure:

CloudWatch Logs: Enabled

Log group:

aws-cloudtrail-logs-986881030460-ec789ed1

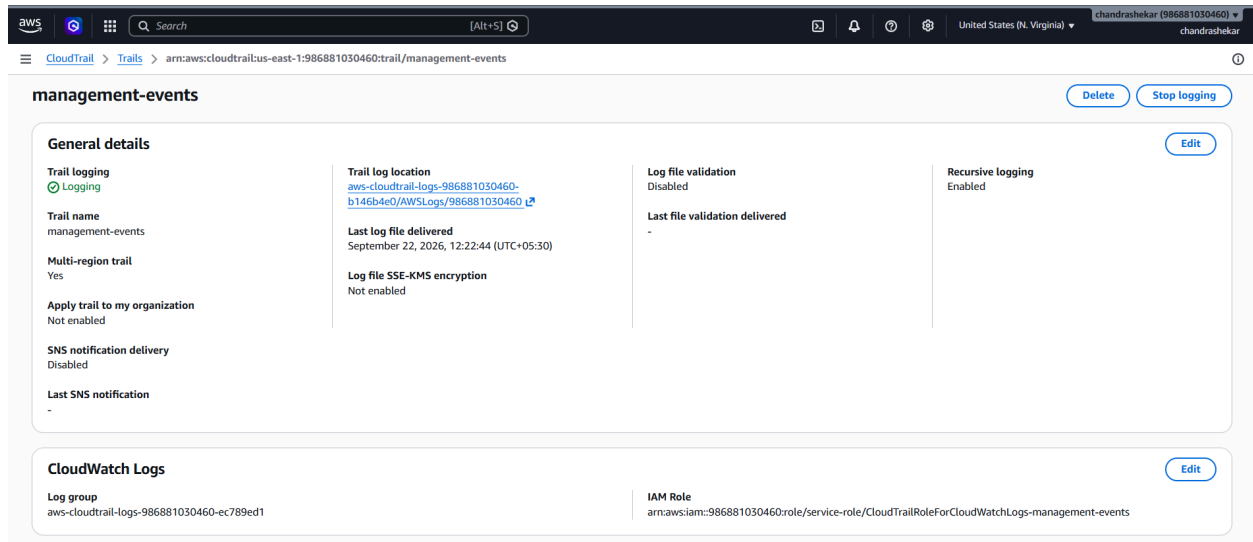
IAM Role:

CloudTrailRoleForCloudWatchLogs-management-events

The IAM role allows CloudTrail to send events to the CloudWatch Logs log group.

Click **Save changes**.

 **Screenshot 3:** Use your screenshot showing the **CloudWatch Logs** section with the Log Group and IAM Role.



Step 4 — Verify the CloudWatch Log Group

Navigate to:

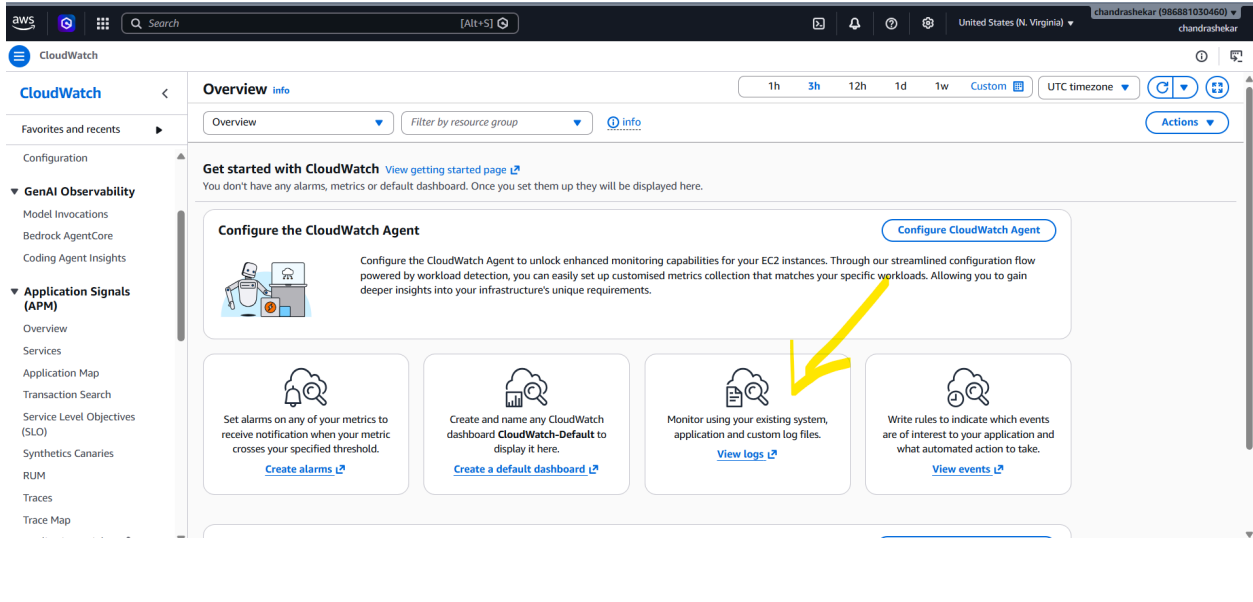
CloudWatch → **Log management** → **Log groups**

Verify that the CloudTrail log group exists.

 **Screenshot 4:** Use your screenshot showing:

aws-cloudtrail-logs-...-ec789ed1

under **Log groups**.



Step 5 — Verify CloudTrail events in CloudWatch

Open:

Log group → Log streams → Latest log stream

CloudTrail events should appear in JSON format.

The events contain information such as:

eventVersion

userIdentity

eventTime

eventSource

eventName

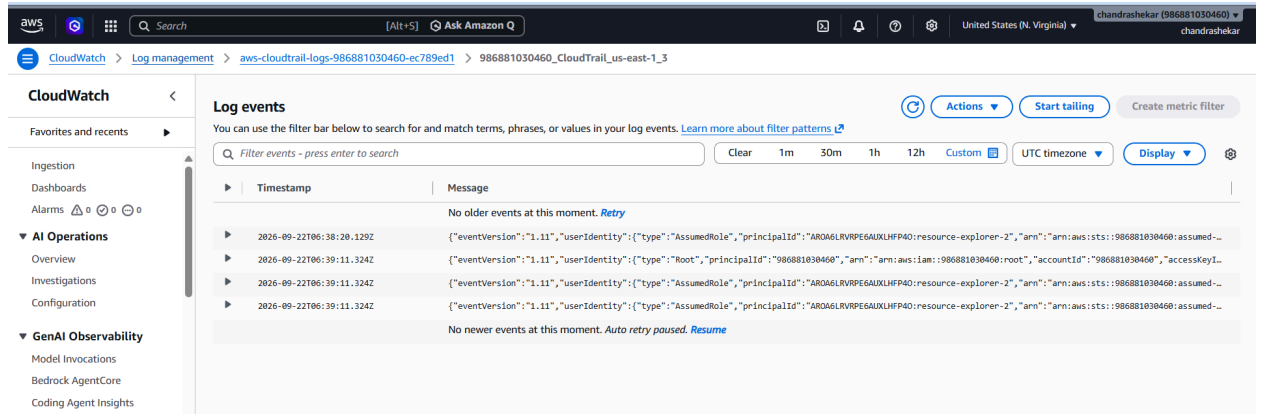
awsRegion

sourceIPAddress

 **Screenshot 5 — IMPORTANT:** Use the screenshot you sent showing multiple **Log events** with JSON messages.

Add this caption:

CloudTrail events successfully delivered to Amazon CloudWatch Logs.



Step 6 — Verify CloudTrail logs in S3

Navigate to the CloudTrail S3 bucket:

S3



CloudTrail bucket



AWSLogs



Account ID



CloudTrail



us-east-1



2026



09



22

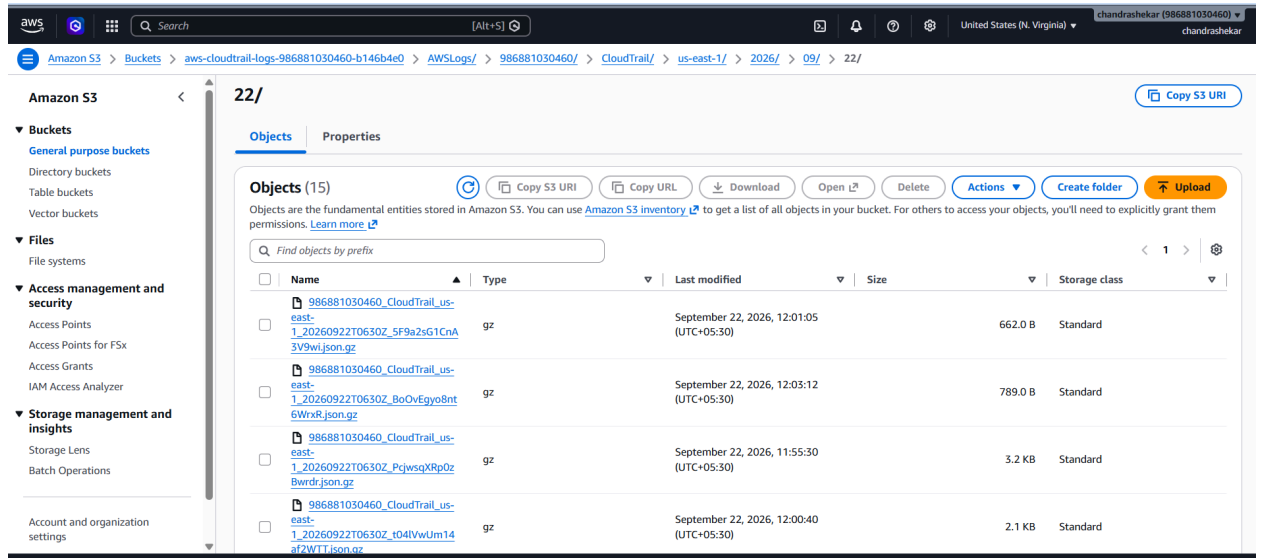
CloudTrail stores the logs as compressed files:

.json.gz

 **Screenshot 6 — IMPORTANT:** Use your latest screenshot showing the `.json.gz` files.

Add this caption:

CloudTrail log files successfully delivered and stored in Amazon S3.



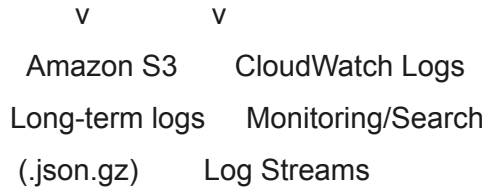
Result

Task 1 completed successfully. AWS CloudTrail was configured to record AWS management activity. CloudTrail logs are being stored in an Amazon S3 bucket for persistent storage and are also being delivered to Amazon CloudWatch Logs for monitoring and analysis.

Architecture to draw

You can make this simple diagram in your document:





2.Enable SNS for CloudTrail to send alerts via email.

Task 2: Configure SNS for CloudTrail Email Notifications

Objective

To configure **Amazon SNS (Simple Notification Service)** with AWS CloudTrail so that email notifications are received when CloudTrail delivers new log files to the configured S3 bucket.

Architecture

You can draw this simple flow:

AWS Account Activity



CloudTrail



Log delivered
to S3



SNS Topic

cloudtrail-notifications



Email Subscription



Gmail

Step 1 — Enable SNS Notification Delivery in CloudTrail

Navigate to:

AWS Console → **CloudTrail** → **Trails** → **management-events** → **Edit**

Under **Additional settings**, enable:

SNS notification delivery: Enabled

Select:

Create a new SNS topic: New

SNS topic:

cloudtrail-notifications

Click **Save changes**.



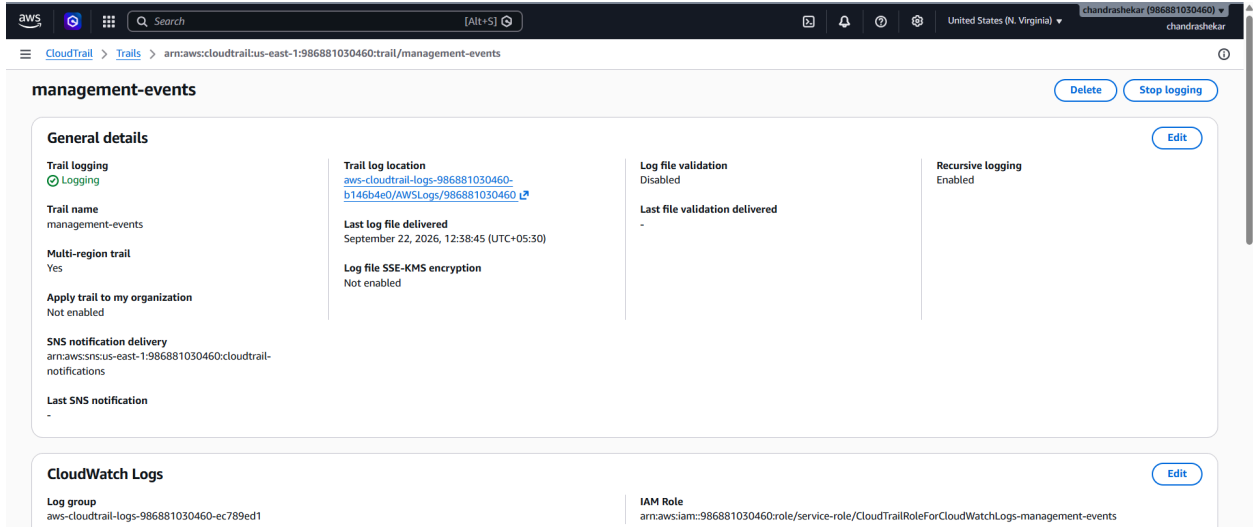
Screenshot 1

Use the screenshot where you enabled:

- SNS notification delivery
- New SNS topic
- **cloudtrail-notifications**

Caption:

SNS notification delivery enabled for the CloudTrail trail and a new SNS topic configured.



Step 2 — Verify SNS Integration with CloudTrail

After saving, open:

CloudTrail → **Trails** → **management-events**

Under **General details**, verify:

SNS notification delivery:

arn:aws:sns:us-east-1:....:cloudtrail-notifications

This confirms that the SNS topic is attached to your CloudTrail trail.

Screenshot 2

Use this screenshot:

Edit arn:aws:cloudtrail:us-west-2:51[redacted]:trail/cloudtrail-events

General details

A trail created in the console is a multi-region trail. [Learn more](#) 

Trail name

Enter a display name for your trail.

cloudtrail-events

3-128 characters. Only letters, numbers, periods, underscores, and dashes are allowed.

☐ Enable for all accounts in my organization

To review accounts in your organization, open AWS Organizations. [See all accounts](#) 

Storage location [Info](#)

☐ Create new S3 bucket
Create a bucket to store logs for the trail.

☒ Use existing S3 bucket
Choose an existing bucket to store logs for this trail.

Trail log bucket name

Enter a new S3 bucket name and folder (prefix) to store your logs. Bucket names must be globally unique.

aws-cloudtrail-logs-51[redacted]106-7c98048d X

Browse

Prefix - optional

prefix

Logs will be stored in aws-cloudtrail-logs-51[redacted]106-7c98048d/AWSLogs/51[redacted].106

Log file SSE-KMS encryption [Info](#)

☐ Enabled

▼ Additional settings

Log file validation [Info](#)

☐ Enabled

SNS notification delivery [Info](#)

☐ Enabled

Cancel

Save changes

How does CloudTrail pricing work?

CloudTrail events can be processed by one trail for free. There is a charge for processing events by additional trails. For more information, see [Pricing](#).

Learn more

[Pricing](#)
[Documentation](#)
[Forums](#)
[FAQs](#)

Create Trail

Trail name*

Apply trail to all regions ☒ Yes ☐ No ⓘ

Create a new S3 bucket ☒ Yes ☐ No

S3 bucket* ⓘ

Log file prefix ⓘ

Location: sampleprefix/AWSLogs/111111111111/CloudTrail/us-west-2

Encrypt log files ☐ Yes ☒ No ⓘ

Enable log file validation ☒ Yes ☐ No ⓘ

Send SNS notification for every log file delivery ☒ Yes ☐ No ⓘ

Create a new SNS topic ☒ Yes ☐ No

SNS topic* ⓘ

* Required field

Additional charges may apply ⓘ

[Cancel](#)

[Create](#)

Prefix - optional

Logs will be stored in aws-cloudtrail-logs- -4a8dcb98/AWSLogs/

Log file SSE-KMS encryption [Info](#)

☐ Enabled

▼ Additional settings

Log file validation [Info](#)

☒ Enabled

SNS notification delivery [Info](#)

☐ Enabled

[Cancel](#)

[Save changes](#)

Actually, in **your document**, use **your own screenshot** where the **management-events** page shows:

SNS notification delivery

arn:aws:sns:us-east-1:....:cloudtrail-notifications

Caption:

SNS topic successfully associated with the CloudTrail trail.

Step 3 — Create Email Subscription

Navigate to:

Amazon SNS → **Topics** → **cloudtrail-notifications**

Click:

Create subscription

Configure:

Protocol: Email

Endpoint: <email-address>

Click **Create subscription**.

Initially, the subscription status will be:

Pending confirmation



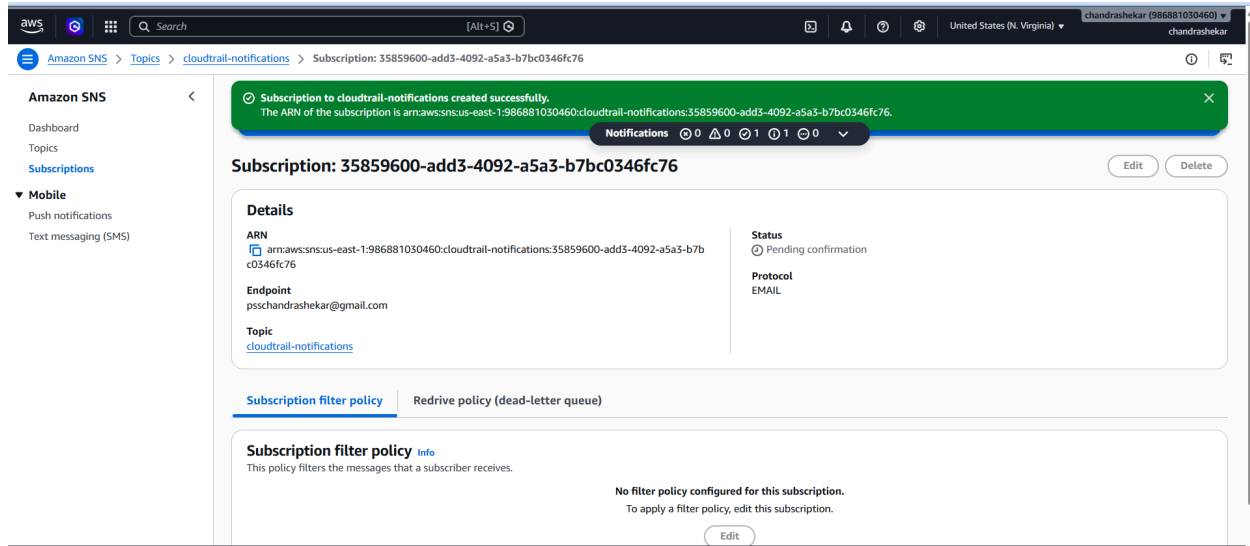
Screenshot 3

Use your screenshot showing:

- Topic: **cloudtrail-notifications**
- Protocol: EMAIL
- Status: **Pending confirmation**

Caption:

Email subscription created for the SNS topic and waiting for confirmation.



Step 4 — Confirm the Email Subscription

Open the confirmation email received from **AWS Notifications**.

Click:

Confirm subscription

AWS displays:

Subscription confirmed!

You have successfully subscribed.



Screenshot 4

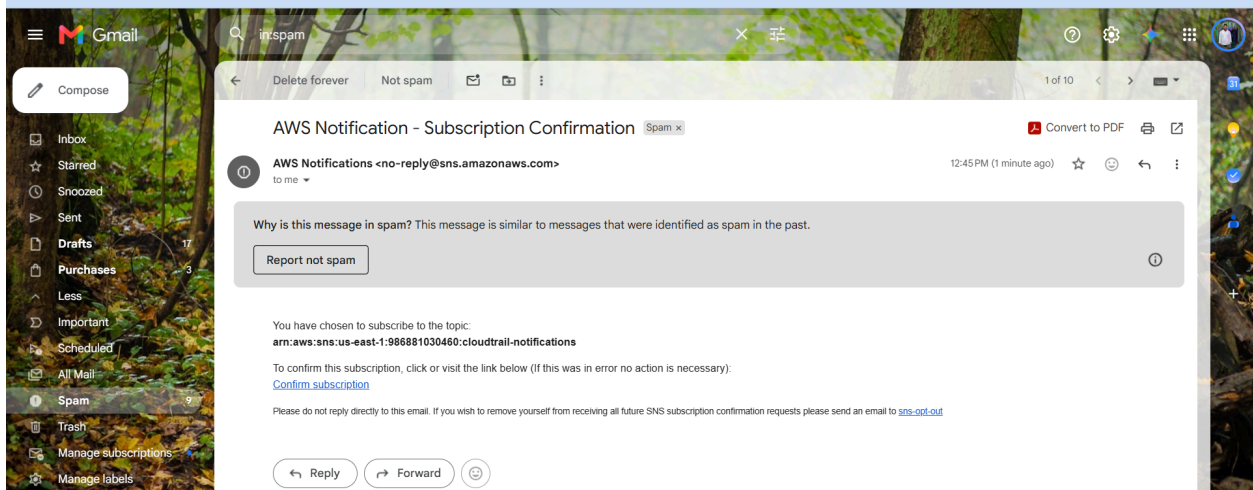
Use your screenshot showing the green:

Subscription confirmed!

message.

Caption:

SNS email subscription successfully confirmed through the AWS confirmation email.



Step 5 — Verify Subscription Status

Return to:

Amazon SNS → Topics → cloudtrail-notifications → Subscriptions

Verify:

Protocol: EMAIL

Status: Confirmed

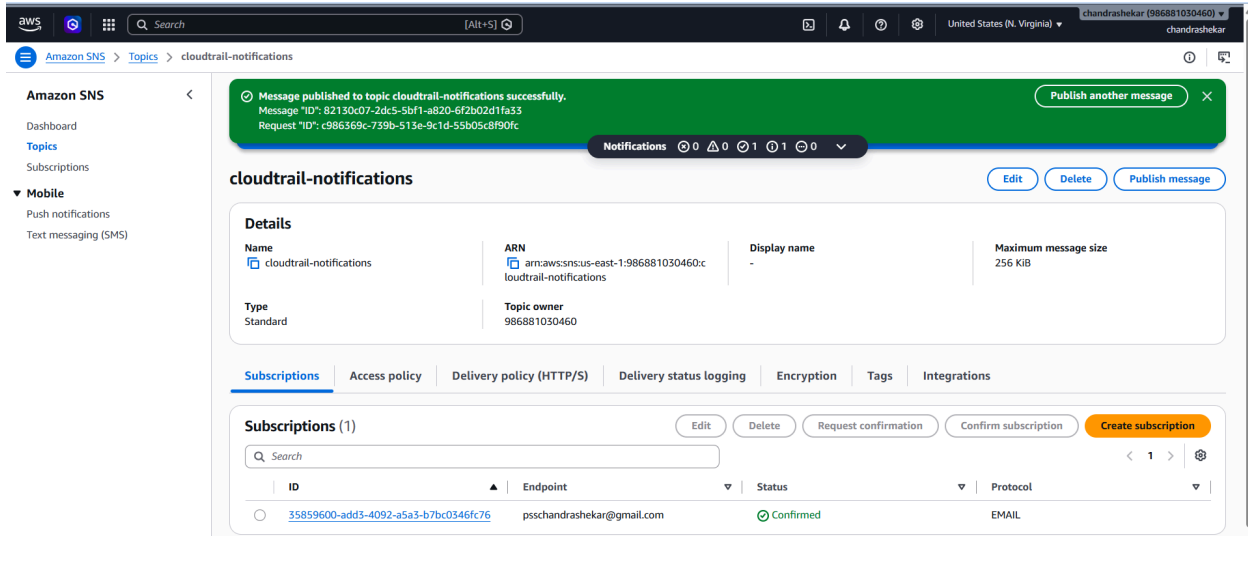


Screenshot 5 — Important

Use your screenshot showing the SNS subscription with the green **Confirmed** status.

Caption:

SNS email subscription status verified as Confirmed.



Step 6 — Test the SNS Topic

Open:

SNS → Topics → cloudtrail-notifications → Publish message

Enter a test subject and message.

Example:

Subject:

CloudTrail SNS Test Alert

Message:

CloudTrail SNS notification test successful.

This message confirms that SNS email notifications are working.

Click **Publish message**.

SNS displays:

Message published to topic

cloudtrail-notifications successfully.

 **Screenshot 6**

Use your screenshot with the green:

Message published to topic cloudtrail-notifications successfully

banner.

Caption:

Test message successfully published to the SNS topic.

Step 7 — Verify Actual CloudTrail Notification in Email

Check Gmail.

You received actual AWS notification messages containing information similar to:

s3Bucket

s3ObjectKey

AWSLogs/

CloudTrail/

.json.gz

This is especially important because it proves that **CloudTrail itself is publishing S3 log-delivery notifications to SNS and SNS is delivering them to email.**



Screenshot 7 — Most Important

Use your Gmail screenshot showing:

AWS Notification Message

and the message containing:

s3Bucket

s3ObjectKey

CloudTrail

.json.gz

Caption:

CloudTrail log delivery notification successfully received through Amazon SNS email subscription.

Final Result

At the end of Task 2, write:

Task 2 completed successfully. Amazon SNS was integrated with the existing CloudTrail trail using the `cloudtrail-notifications` topic. An email subscription was created and confirmed. SNS publishing was tested successfully, and actual CloudTrail S3 log-delivery notifications were received through email.

Important interview note

Write this as a small **Note**:

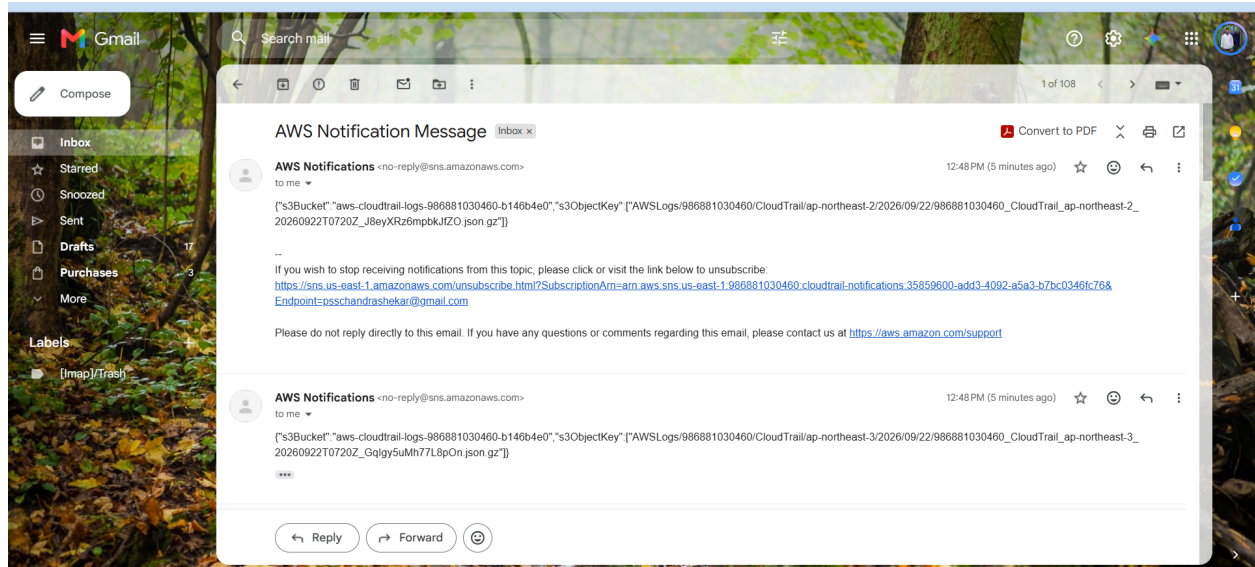
CloudTrail SNS notifications indicate the delivery of CloudTrail log files to S3. They do not represent an individual email alert for every AWS API event. Specific API-event alerts can be implemented using services such as CloudWatch/EventBridge together with SNS.

Screenshots to submit

You have **7 possible screenshots**, but if you want a shorter professional document, use only **5**:

1. CloudTrail — SNS notification enabled.
2. CloudTrail — `cloudtrail-notifications` topic attached.
3. SNS — email subscription **Confirmed**.
4. SNS — **Message published successfully**.
5. Gmail — actual **AWS CloudTrail notification received**. 

The **last screenshot is the strongest proof that Task 2 is working end-to-end**.



3. Configure CloudWatch monitoring and record the CPU utilization and other metrics of EC2.

.Task 3: Monitor EC2 Memory and Disk Utilization Using CloudWatch Agent

Objective:

To configure the Amazon CloudWatch Agent on an EC2 instance to collect operating-system-level metrics such as memory and disk utilization.

Steps Performed:

1. Launched an Ubuntu EC2 instance named **cloudwatch-monitoring-server**.
2. Created an IAM role named **EC2-CloudWatch-Agent-Role** with the required CloudWatch Agent permissions.
3. Attached the IAM role to the EC2 instance.
4. Connected to the EC2 instance using SSH.
5. Downloaded and installed the Amazon CloudWatch Agent.
6. Ran the CloudWatch Agent configuration wizard and enabled host-level metrics.
7. Started the CloudWatch Agent using the generated configuration.

Verified the agent status and confirmed:

status: running

8. configstatus: configured
9. Opened **CloudWatch** → **Metrics** → **Classic metrics** → **CWAgent**.
10. Selected **mem_used_percent** to monitor memory utilization.
11. Selected **disk_used_percent** to monitor disk utilization.
12. Verified that both metrics were successfully reporting data to Amazon CloudWatch.

Where to put your screenshots

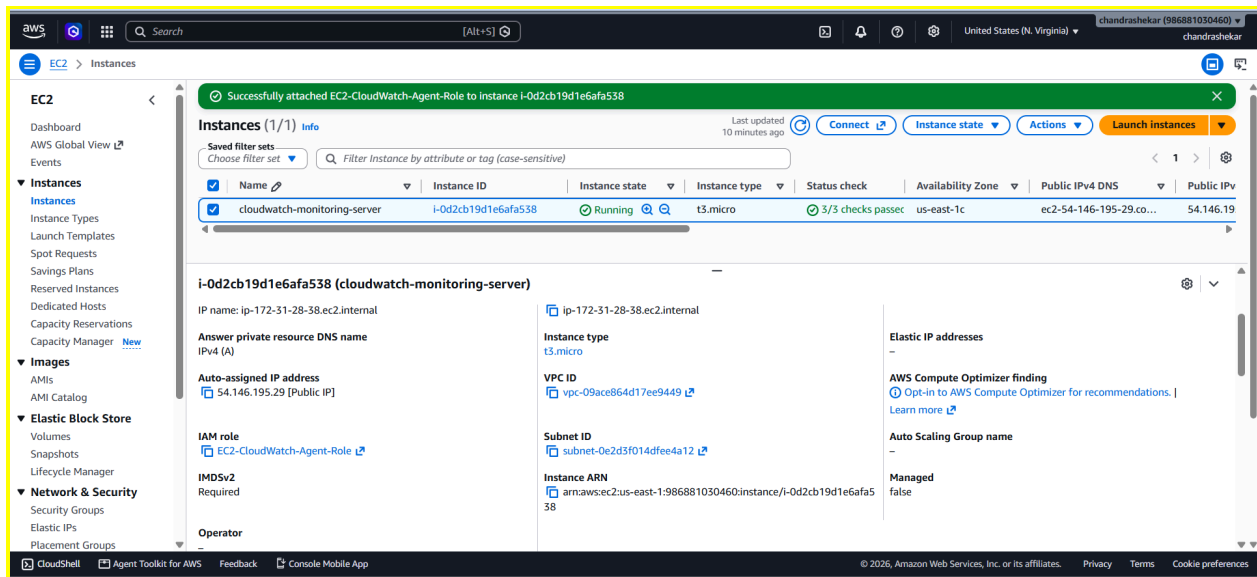
You don't need every screenshot you took. Use the strongest evidence in this order:

Screenshot 1 – EC2 Instance Created

Put your screenshot showing **cloudwatch-monitoring-server**, **Running**, instance ID and instance type.

Caption:

Figure 1: EC2 instance created for CloudWatch monitoring.

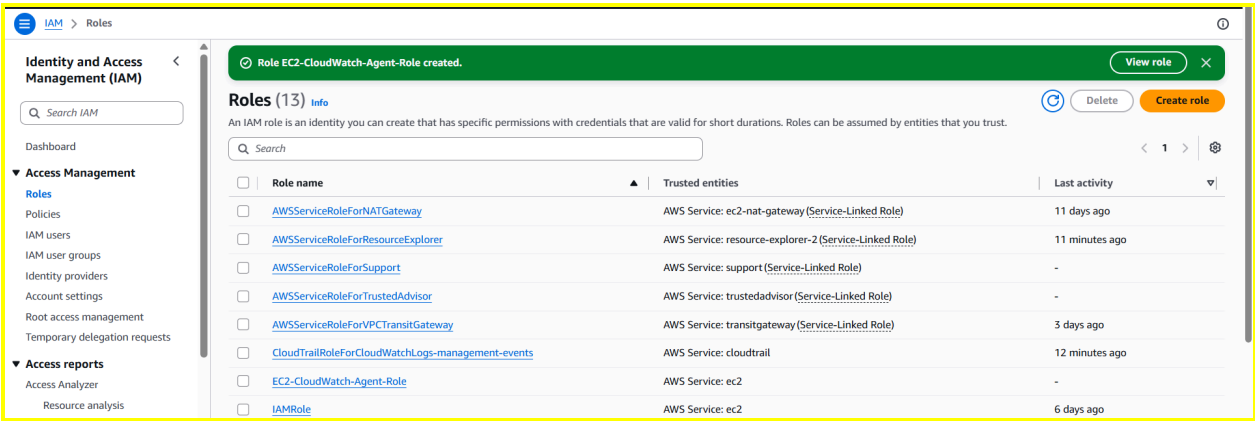


Screenshot 2 – IAM Role Created

Use the screenshot showing **EC2-CloudWatch-Agent-Role** created.

Caption:

Figure 2: IAM role created to provide CloudWatch Agent permissions.



Screenshot 3 – IAM Role Attached to EC2

Use the screenshot with the green message:

Successfully attached EC2-CloudWatch-Agent-Role...

Caption:

Figure 3: CloudWatch Agent IAM role successfully attached to the EC2 instance.

ubuntu@ip-172-31-28-38: ~

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)

\$ chmod 400 "my_server.pem"

HP@DESKTOP-1GQ065F MINGW64 ~/Downloads (master)

\$ ssh -i "my_server.pem" ubuntu@ec2-54-146-195-29.compute-1.amazonaws.com

The authenticity of host 'ec2-54-146-195-29.compute-1.amazonaws.com' (54.146.195.29) can't be established.

ED25519 key fingerprint is SMA256:Rb2Y2IIP+1qLEyResDxt/XkheFLLmJiawGrr1x7poGA.

This key is not known by any other names

Are you sure you want to continue connecting (yes/no/[fingerprint])? yes

Warning: Permanently added 'ec2-54-146-195-29.compute-1.amazonaws.com' (ED25519) to the list of known hosts.

Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

* Documentation: <https://docs.ubuntu.com>
* Management: <https://landscape.canonical.com>
* Support: <https://ubuntu.com/pro>

System information as of Tue Sep 22 09:44:18 UTC 2026

System load:	0.0	Temperature:	-273.1 C
Usage of /:	30.5% of 6.61GB	Processes:	116
Memory usage:	27%	Users logged in:	0
Swap usage:	0%	IPv4 address for ens5:	172.31.28.38

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See <https://ubuntu.com/esm> or run: `sudo pro status`

The list of available updates is more than a week old.
To check for new updates run: `sudo apt update`

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in `/usr/share/doc/*/copyright`.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-28-38:~\$ sudo apt update

```
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 Packages [16.3 MB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64v3 Packages [502 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [117 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/main amd64 c-n-f Metadata [4616 B]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/universe amd64v3 Packages [174 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [54.8 kB]
Get:12 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.4 kB]
Get:13 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 c-n-f Metadata [3552 B]
Get:14 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64v3 Packages [418 kB]
```

Received 34.8 MB in 6 (6070 K/s)

181 packages can be upgraded. Run 'apt list --upgradable' to see them.

ubuntu@ip-172-31-28-38:~\$ uname -m

x86_64

ubuntu@ip-172-31-28-38:~\$ wget https://amazoncloudwatch-agent.s3.amazonaws.com/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb

--2026-09-22 09:45:11-- https://amazoncloudwatch-agent.s3.amazonaws.com/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb

Resolving amazoncloudwatch-agent.s3.amazonaws.com (amazoncloudwatch-agent.s3.amazonaws.com)... 16.15.238.14, 54.231.193.65, 52.217.231.57, ...

Connecting to amazoncloudwatch-agent.s3.amazonaws.com (amazoncloudwatch-agent.s3.amazonaws.com)[16.15.238.14]:443... connected.

HTTP request sent, awaiting response... 200 OK

length: 69937052 (67M) [application/octet-stream]

Saving to: 'amazon-cloudwatch-agent.deb'

amazon-cloudwatch-agent.deb

100%[=====] 66.70M 98.8MB/s in 0.7

2026-09-22 09:45:12 (98.8 MB/s) - 'amazon-cloudwatch-agent.deb' saved [69937052/69937052]

ubuntu@ip-172-31-28-38:~\$ sudo dpkg -i amazon-cloudwatch-agent.deb

Selecting previously unselected package amazon-cloudwatch-agent.

(Reading database ... 85303 files and directories currently installed.)

Preparing to unpack amazon-cloudwatch-agent.deb ...

create group cwagent, result: 0

create user cwagent, result: 0

Unpacking amazon-cloudwatch-agent (1.300073.1b1859-1) ...

Setting up amazon-cloudwatch-agent (1.300073.1b1859-1) ...

ubuntu@ip-172-31-28-38:~\$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status

{

"status": "stopped",

"starttime": "",

"configstatus": "not configured",

"version": "1.300073.1b1859"

}
ubuntu@ip-172-31-28-38:~\$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard

Starting config-wizard, this will map back to a call to amazon-cloudwatch-agent

Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-wizard]

Welcome to the Amazon Cloudwatch Agent Configuration Manager

= Cloudwatch Agent allows you to collect metrics and logs from =

= your host and send them to Cloudwatch. Additional Cloudwatch =

= charges may apply. =

=====

On which OS are you planning to use the agent?

1. linux

2. windows

3. darwin

default choice: [1]:

Advanced

The value Advanced is not valid to this question.

Please retry to answer:

On which OS are you planning to use the agent?

1. linux

2. windows

3. darwin

default choice: [1]:

1

```
ubuntu@ip-172-31-28-38:~$ }
}
Please check the above content of the config.
The config file is also located at /opt/aws/amazon-cloudwatch-agent/bin/config.json.
Edit it manually if needed.
Do you want to store the config in the SSM parameter store?
1. yes
2. no
Default choice: [1]:
no
The value no is not valid to this question.
Please retry to answer:
Do you want to store the config in the SSM parameter store?
1. yes
2. no
Default choice: [1]:
2
Program exits now.
ubuntu@ip-172-31-28-38:~$ /opt/aws/amazon-cloudwatch-agent/bin/config.json
-bash: /opt/aws/amazon-cloudwatch-agent/bin/config.json: Permission denied
ubuntu@ip-172-31-28-38:~$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-a fetch-config \
-m ec2 \
-c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json \
-s
***** processing amazon-cloudwatch-agent *****
Starting config-downloader, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-downloader -multi-config default -mode ec2 -download-source file:/opt/aws/amazon-cloudwatch-agent/bin/config.json -
output-dir /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d -config /opt/aws/amazon-cloudwatch-agent/etc/common-config.toml]!! Trying to detect region from ec2
D! [EC2] Found active network interface
I! imds retry client will retry 1 times
Start configuration validation...
2026/09/22 09:59:45 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d/file_config.json.tmp ...
2026/09/22 09:59:45 I! Valid json input schema.
2026/09/22 09:59:45 D! ec2tagger processor required because append_dimensions is set
2026/09/22 09:59:45 D! delta processor required because metrics with diskio or net are set
2026/09/22 09:59:45 D! ec2tagger processor required because append_dimensions is set
2026/09/22 09:59:45 Configuration validation first phase succeeded
Starting config-translator, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-translator -mode ec2 -config /opt/aws/amazon-cloudwatch-agent/etc/common-config.toml -multi-config default -input /
/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json -input-dir /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d -output /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.to
m]!! Trying to detect region from ec2
D! [EC2] Found active network interface
I! imds retry client will retry 1 times
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -schematest -config /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml
Configuration validation second phase succeeded
Configuration validation succeeded
amazon-cloudwatch-agent has already been stopped
Created symlink /etc/systemd/system/multi-user.target.wants/amazon-cloudwatch-agent.service' -> /etc/systemd/system/amazon-cloudwatch-agent.service'.
ubuntu@ip-172-31-28-38:~$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status
{
  "status": "running",
  "starttime": "2026-09-22T09:59:46+00:00",
  "configstatus": "configured",
  "version": "1.300073.1b1859"
}
ubuntu@ip-172-31-28-38:~$ }
```

Screenshot 4 – Agent Installation and Status

Take/use the terminal screenshot where this is clearly visible:

`sudo /opt/aws/amazon-cloudwatch-agent/bin/download-multi-cloudwatch-agent-ctl -a status`

and:

`"status": "running"`

`"configstatus": "configured"`

Caption:

Figure 4: Amazon CloudWatch Agent successfully configured and running.

Screenshot 5 – Memory Metric

👉 Use your first screenshot from this message.

It clearly shows:

`mem_used_percent`

Caption:

Figure 5: Memory utilization of the EC2 instance monitored using the CWAgent `mem_used_percent` metric.

Screenshot 6 – Memory + Disk Final Verification

👉 Use your **second screenshot** from this message. This is your best final screenshot.

It shows:

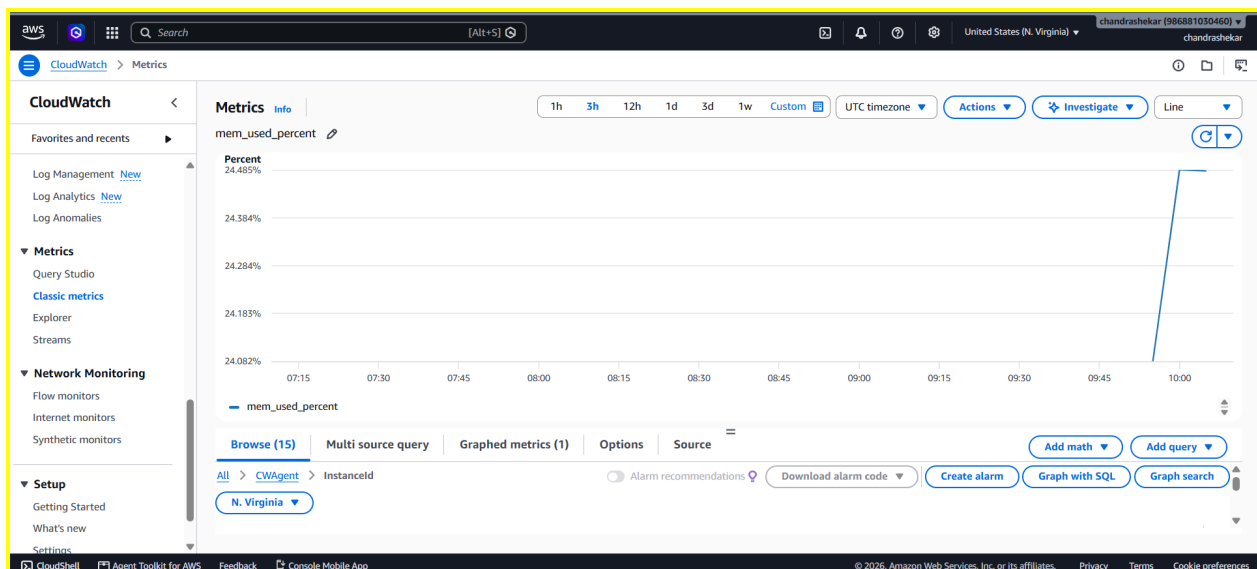
mem_used_percent

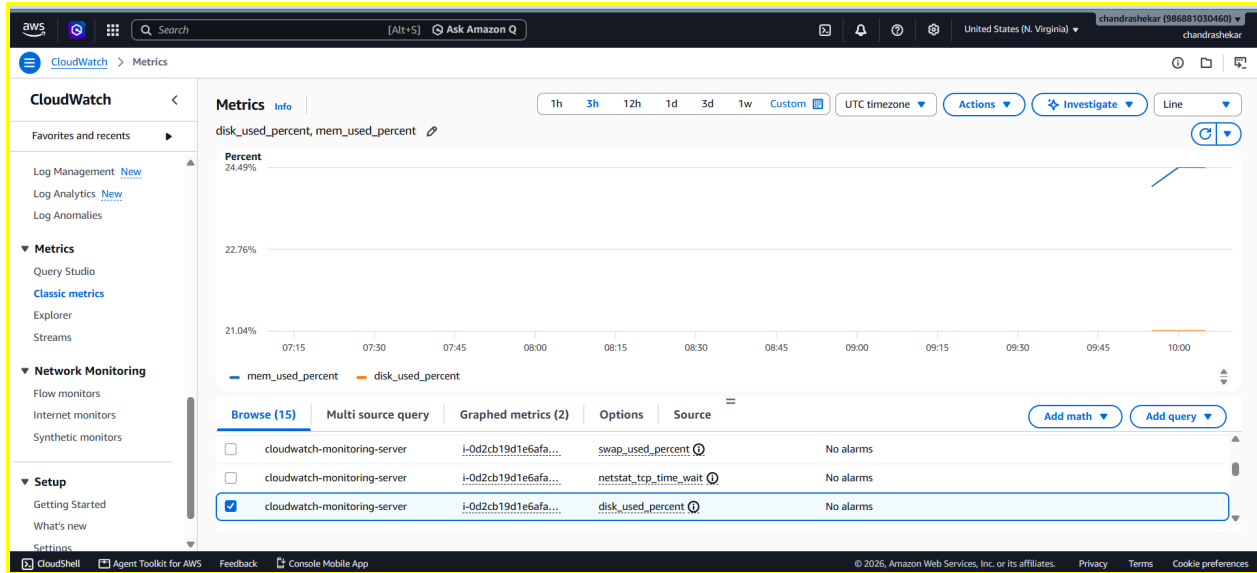
disk_used_percent

and the graph.

Caption:

Figure 6: Successful monitoring of memory and disk utilization using Amazon CloudWatch Agent.





Result

At the end of Task 3, write:

Result: Amazon CloudWatch Agent was successfully installed and configured on the Ubuntu EC2 instance. The agent is running and publishing custom host-level metrics to the **CWAgent** namespace. Memory utilization (**mem_used_percent**) and disk utilization (**disk_used_percent**) were successfully verified in the Amazon CloudWatch console.

So Task 3 is completed successfully. ✓

4.Create one alarm to send an alert to email if the CPU utilization is more than 70 percent.

Task 4: Create a CloudWatch Alarm for CPU Utilization Above 70%

Configured an Amazon CloudWatch alarm to monitor the CPU utilization of the EC2 instance. The alarm threshold was configured as **CPUUtilization > 70%** with an evaluation period of **5 minutes**. An Amazon SNS topic was configured as the notification action.

To test the alarm, CPU load was generated on the EC2 instance using:

```
stress --cpu 2 --timeout 600
```

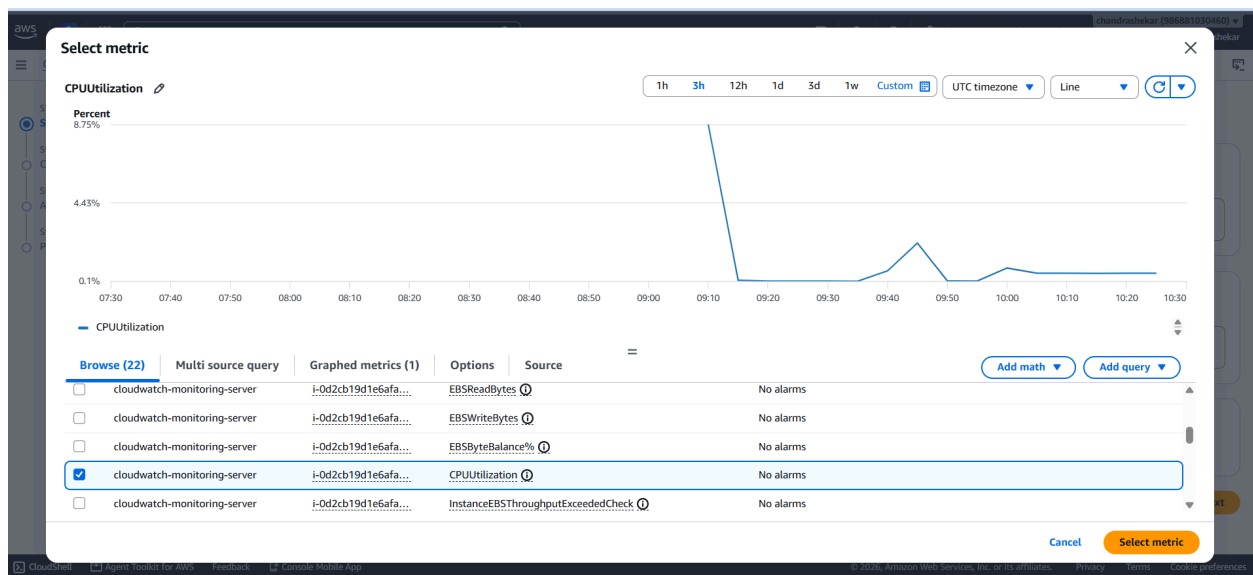
The CPU utilization increased to approximately **99.99%**, causing the CloudWatch alarm to transition from **OK** → **ALARM**. Amazon SNS then successfully sent the alarm notification to the configured email address.

 Put this Gmail screenshot at the end of Task 4 with this caption:

Figure: CloudWatch CPU alarm successfully triggered after CPU utilization exceeded 70%, and the alert was delivered through Amazon SNS email notification.

This is strong final evidence that the complete flow worked:

EC2 CPU → CloudWatch Metric → 70% Alarm → SNS → Email 



us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#alarmsV2:create?~(Page~MetricSelection~metricType~classic~AlarmType~MetricAlarm~AlarmData~(Ala...

CloudWatch > Alarms > Create alarm

InstanceId: i-0d2cb19d1e6afa538 Instance name: cloudwatch-monitoring-server

Statistic: Average Period: 5 minutes

Preview

Graph

This alarm will trigger when the blue line goes above the red line for 1 datapoints within 5 minutes.

Percent

70% 35.05% 0.1%

08:00 08:30 09:00 09:30 10:00 10:30

CPUUtilization

Static

Use a value as a threshold

Anomaly detection

Use a band as a threshold

Whenever CPUUtilization is...

Define the alarm condition.

Greater

> threshold

Greater/Equal

>= threshold

Lower/Equal

<= threshold

Lower

< threshold

than...

Define the threshold value.

70

Must be a number.

Additional configuration

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

CloudWatch > Alarms > Create alarm

Step 1: Specify metric and conditions

Step 2: Configure actions

Step 3: Add alarm details

Step 4: Preview and create

Configure actions

Notification

Alarm state trigger

Define the alarm state that will trigger this action.

In alarm

The metric or expression is outside of the defined threshold.

OK

The metric or expression is within the defined threshold.

Insufficient data

The alarm has just started or not enough data is available.

Remove

Send a notification to the following SNS topic

Define the SNS (Simple Notification Service) topic that will receive the notification.

Select an existing SNS topic

Create new topic

Use topic ARN to notify other accounts

Send a notification to...

cloudtrail-notifications

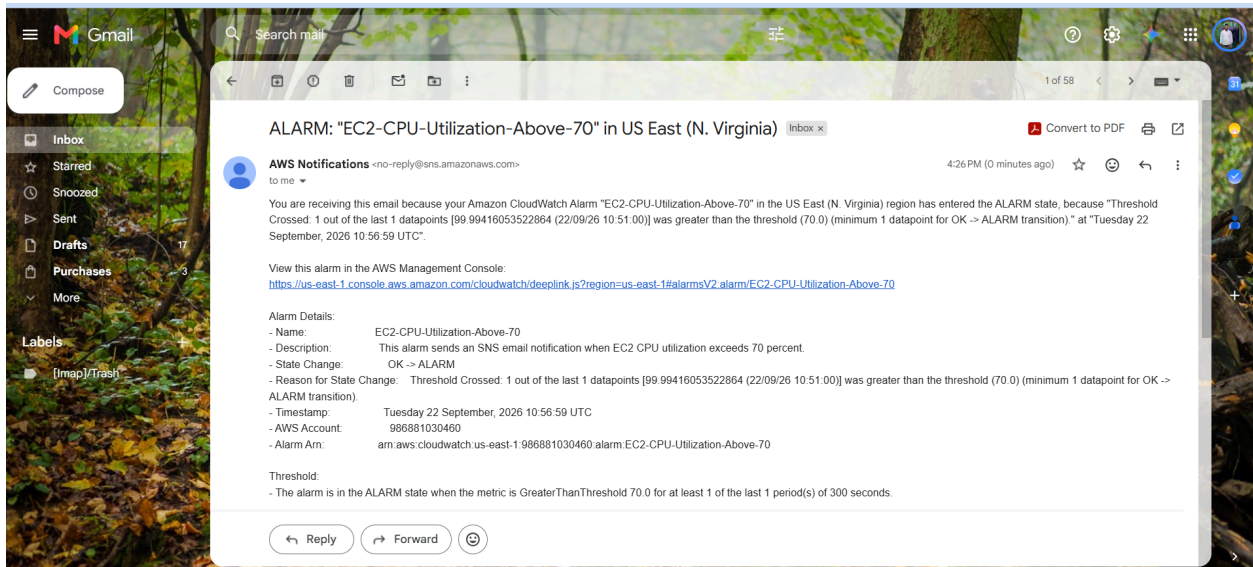
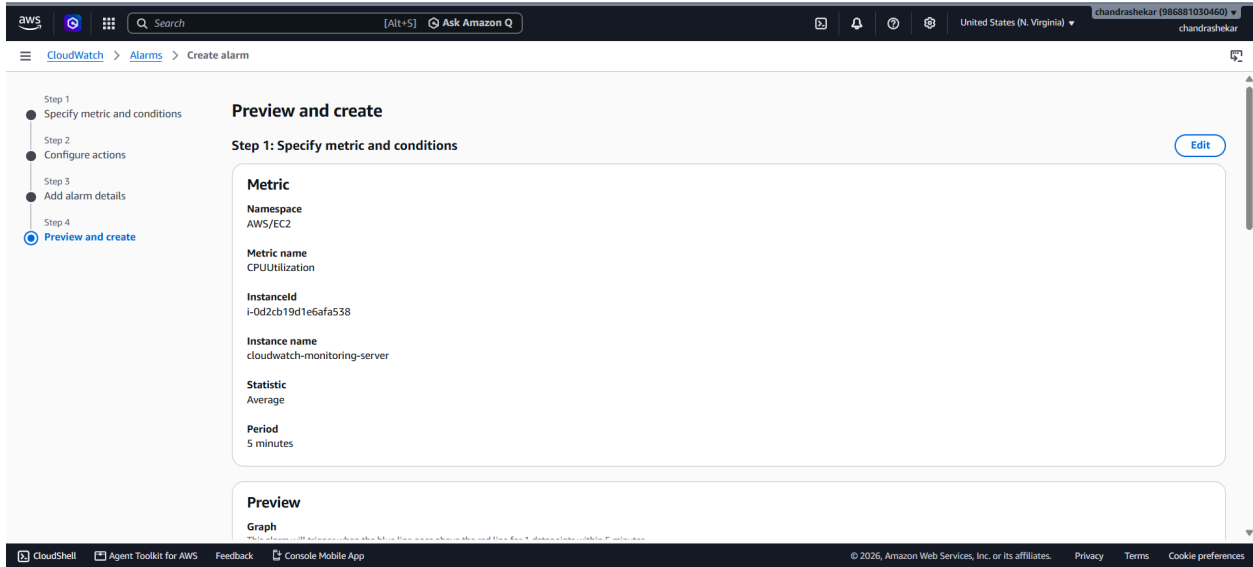
Only topics belonging to this account are listed here. All persons and applications subscribed to the selected topic will receive notifications.

Email (endpoints)

psschandrashekar@gmail.com - View in SNS Console

Add notification

Lambda action



5.Create a Dashboard and monitor the Tomcat service whether it is running or not and send the alert.

Task 5 — Monitor Tomcat Service Using CloudWatch Dashboard & Send Email Alert

Requirement: Create a Dashboard, monitor whether the Tomcat service is running or not, and send an email alert when Tomcat goes down.

1. Verify Tomcat service

On the Ubuntu EC2 instance:

```
sudo systemctl status tomcat10
```

We confirmed:

Active: active (running)

Also verify Tomcat port:

```
sudo ss -ltnp | grep 8080
```

Test Tomcat:

```
curl -I http://localhost:8080
```

Expected:

HTTP/1.1 200

2. Verify AWS CLI

```
aws --version
```

We confirmed AWS CLI v2 was installed.

The EC2 instance must have permission to publish CloudWatch metrics. In your setup, the required IAM permissions were already working because the custom metric successfully reached CloudWatch.

3. Create Tomcat monitoring script

Create:

```
sudo nano /usr/local/bin/tomcat-monitor.sh
```

Use a script like:

```
#!/bin/bash
```

```
if systemctl is-active --quiet tomcat10
```

then

STATUS=1

else

STATUS=0

fi

aws cloudwatch put-metric-data \

--namespace "Custom/Tomcat" \

--metric-name "TomcatServiceStatus" \

--value \$STATUS \

--unit Count \

--region us-east-1

echo "TomcatServiceStatus = \$STATUS"

Meaning:

1 = Tomcat running

0 = Tomcat stopped

Give execute permission:

sudo chmod +x /usr/local/bin/tomcat-monitor.sh

Test:

sudo /usr/local/bin/tomcat-monitor.sh

When Tomcat is running:

TomcatServiceStatus = 1

4. Configure cron

Open root crontab:

sudo crontab -e

Add:

* * * * * /usr/local/bin/tomcat-monitor.sh >> /var/log/tomcat-monitor.log 2>&1

This checks Tomcat **every minute**.

Verify:

```
sudo crontab -l
```

Check output:

```
sudo tail -5 /var/log/tomcat-monitor.log
```

5. Verify custom metric in CloudWatch

Go to:

AWS Console → CloudWatch → Metrics → All/Classic metrics

Select:

Custom namespaces → Custom/Tomcat → Metrics with no dimensions

You should see:

TomcatServiceStatus

Select it.

When Tomcat is running, the metric reports:

1

6. Create CloudWatch Dashboard

Go to:

CloudWatch → Dashboards → Create dashboard

Dashboard name:

Tomcat-Monitoring-Dashboard

Add a widget and select:

CloudWatch → Metrics → Number

Then select:

Custom/Tomcat



TomcatServiceStatus

Create the widget and save the dashboard.

Note: A Number widget can show something such as **0.8** if it averages historical **0** and **1** datapoints over the selected time range. That doesn't mean Tomcat is "80% running."

7. Create Tomcat CloudWatch Alarm

Select the **TomcatServiceStatus** metric and choose **Create alarm**.

Configure:

Metric: TomcatServiceStatus

Namespace: Custom/Tomcat

Statistic: Minimum

Period: 1 minute

Threshold: Static

Condition: Lower than

Value: 1

The logic is:

$1 < 1 \rightarrow \text{FALSE} \rightarrow \text{Tomcat running} \rightarrow \text{OK}$

$0 < 1 \rightarrow \text{TRUE} \rightarrow \text{Tomcat stopped} \rightarrow \text{ALARM}$

8. Configure SNS notification

Under **Configure actions**:

Alarm state trigger: In alarm

Notification: SNS topic

Select your existing SNS topic containing your confirmed email subscription.

The flow is:

CloudWatch Alarm



SNS Topic



Email

9. Give alarm name

Alarm name:

Tomcat-Service-Down-Alarm

Description:

This alarm sends an SNS email notification when the Tomcat10 service is stopped or not running.

Review everything and click **Create alarm**.

10. Test Tomcat failure

Stop Tomcat intentionally:

```
sudo systemctl stop tomcat10
```

Verify:

```
sudo systemctl status tomcat10
```

The monitoring script detects the failure and publishes:

```
TomcatServiceStatus = 0
```

Check:

```
sudo tail -5 /var/log/tomcat-monitor.log
```

CloudWatch then evaluates:

```
0 < 1
```

Therefore:

OK → ALARM 🚨

You successfully received the SNS email:

ALARM: "Tomcat-Service-Down-Alarm"

State Change: OK -> ALARM

Threshold Crossed: 0.0 was less than threshold 1.0

11. Restart Tomcat

```
sudo systemctl start tomcat10
```

Verify:

```
sudo systemctl status tomcat10
```

You got:

Active: active (running)

The monitoring script then published:

TomcatServiceStatus = 1

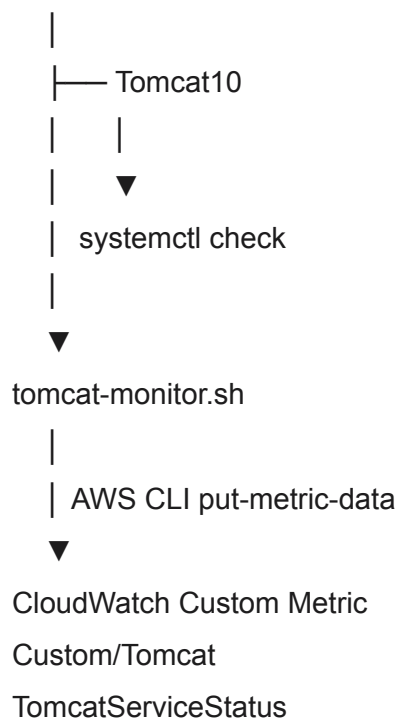
Verify:

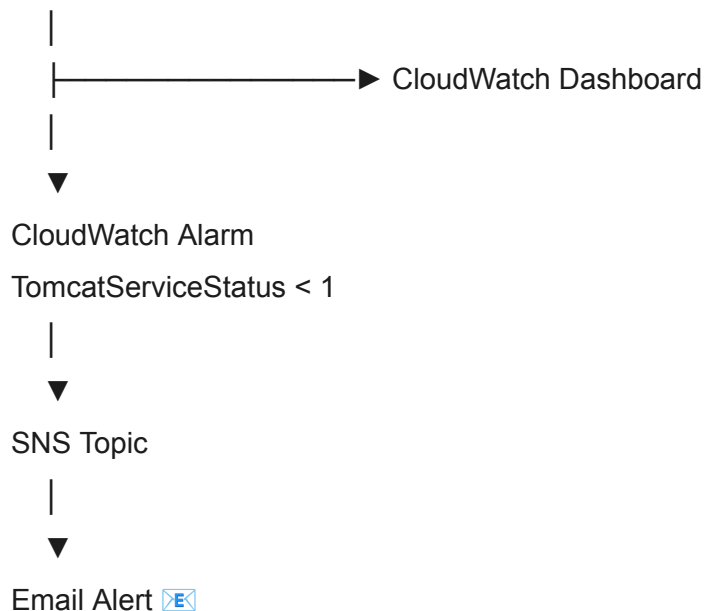
```
sudo tail -5 /var/log/tomcat-monitor.log
```

Eventually the alarm returns from **ALARM** → **OK**.

Final architecture

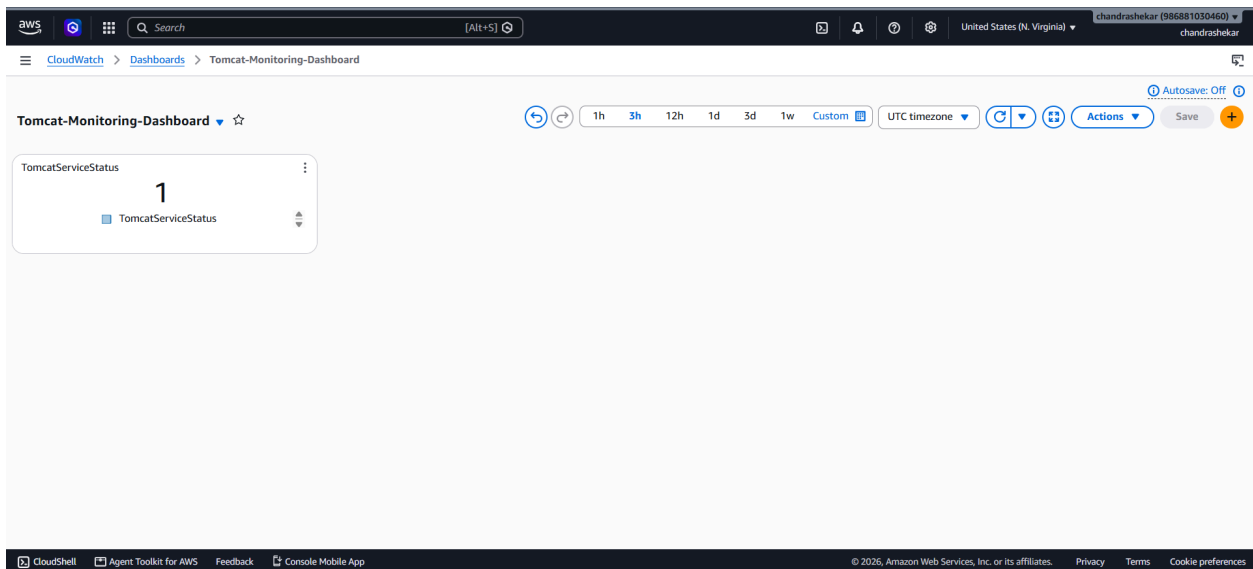
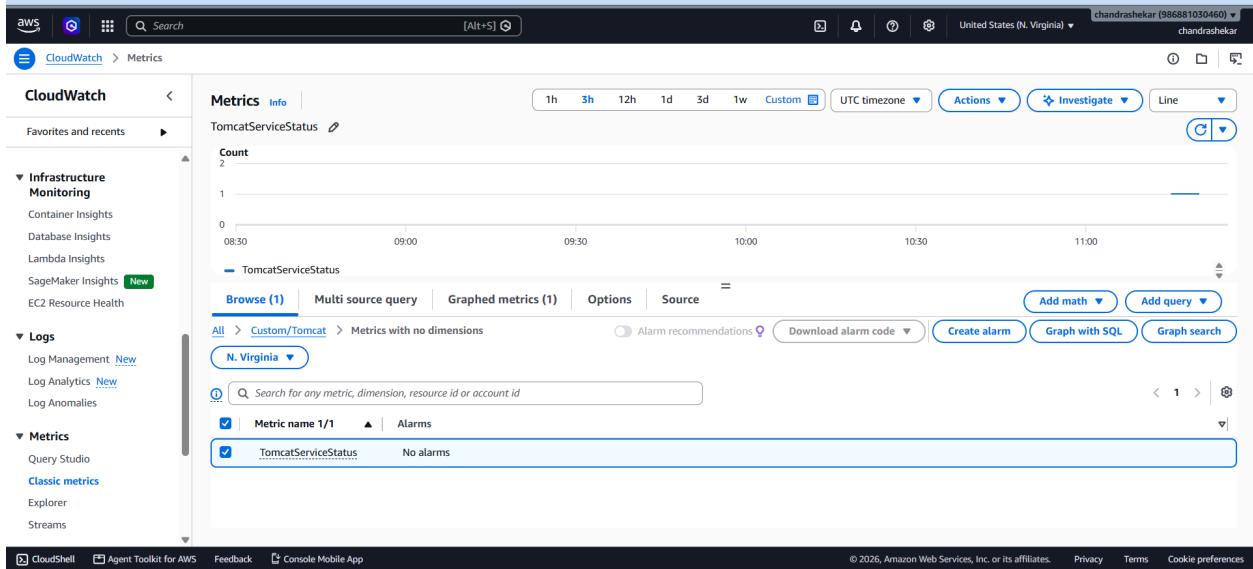
EC2 Ubuntu

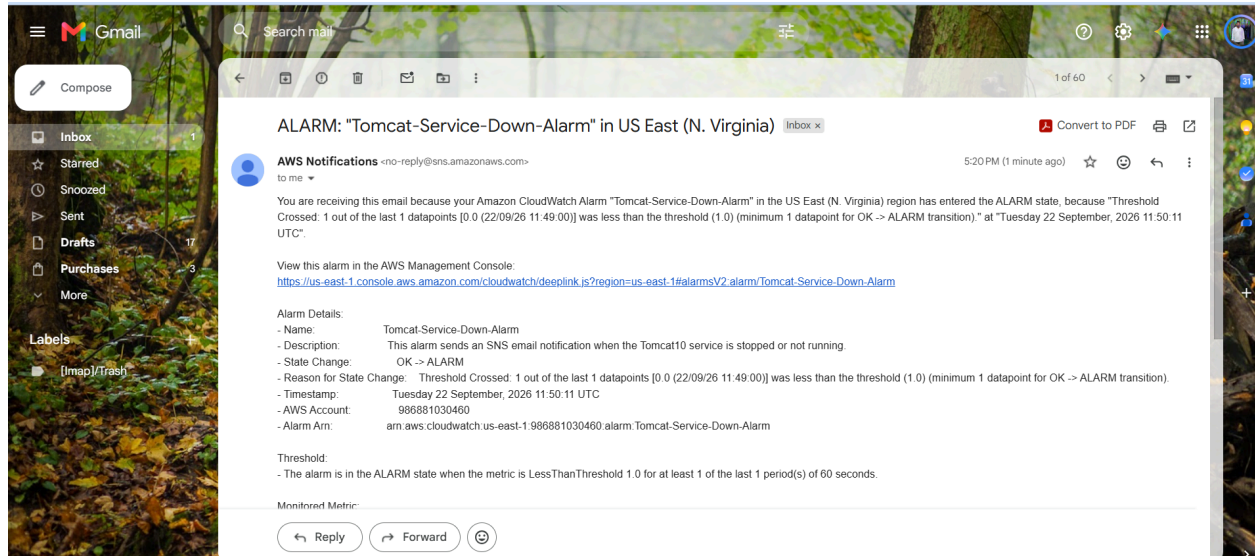




One-line explanation for your trainer/interview: “I created a shell script that checks the Tomcat10 service every minute using cron. It publishes **1** for running and **0** for stopped as a custom CloudWatch metric. I displayed that metric on a CloudWatch dashboard and configured an alarm for values below 1. The alarm is integrated with SNS, so when Tomcat goes down, an email notification is sent automatically.

```
ubuntu@ip-172-31-28-38: ~  
Loaded: loaded (/usr/lib/systemd/system/tomcat10.service; enabled; preset: enabled)  
Active: active (running) since Tue 2026-09-22 11:03:56 UTC; 23s ago  
Invocation: 84365ff827bf40699886237ba0e6f70a  
Docs: https://tomcat.apache.org/tomcat-10.0-doc/index.html  
Main PID: 4238 (java)  
Tasks: 29 (limit: 627)  
Memory: 101.9M (peak: 101.9M)  
CPU: 5.881s  
CGroup: /system.slice/tomcat10.service  
└─4238 /usr/lib/jvm/default-java/bin/java -Djava.util.logging.config.file=/var/lib/tomcat10/conf/logging.properties -Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager -Djava.awt.headless=true  
Sep 22 11:03:57 ip-172-31-28-38 tomcat10[4238]: OpenSSL successfully initialized [OpenSSL 3.5.5 27 Jan 2026]  
Sep 22 11:03:58 ip-172-31-28-38 tomcat10[4238]: Initializing ProtocolHandler ["http-nio-8080"]  
Sep 22 11:03:58 ip-172-31-28-38 tomcat10[4238]: Server initialization in [1637] milliseconds  
Sep 22 11:03:58 ip-172-31-28-38 tomcat10[4238]: Starting service [Catalina]  
Sep 22 11:03:58 ip-172-31-28-38 tomcat10[4238]: Starting Servlet engine: [Apache Tomcat/10.1.55 (Ubuntu)]  
Sep 22 11:03:58 ip-172-31-28-38 tomcat10[4238]: Deploying web application directory [/var/lib/tomcat10/webapps/ROOT]  
Sep 22 11:04:01 ip-172-31-28-38 tomcat10[4238]: At least one JAR was scanned for TLDs yet contained no TLDs. Enable debug logging for this logger for a complete list of JARs that were scanned but no TLDs were  
Sep 22 11:04:01 ip-172-31-28-38 tomcat10[4238]: Deployment of web application directory [/var/lib/tomcat10/webapps/ROOT] has finished in [2,691] ms  
Sep 22 11:04:01 ip-172-31-28-38 tomcat10[4238]: Starting ProtocolHandler ["http-nio-8080"]  
Sep 22 11:04:01 ip-172-31-28-38 tomcat10[4238]: Server startup in [2889] milliseconds  
[lines 1-22/22 (END)]  
[1]+ Stopped sudo systemctl status tomcat10  
ubuntu@ip-172-31-28-38:~$ sudo ss -ltnp | grep 8080  
LISTEN 0 100 *:* 8080 *:* users:((("java",pid=4238,fd=37))  
ubuntu@ip-172-31-28-38:~$ curl -I http://localhost:8080  
HTTP/1.1 200  
Accept-Ranges: bytes  
Etag: w/"1905-1790075035844"  
Last-Modified: Tue, 22 Sep 2026 11:03:55 GMT  
Content-Type: text/html  
Content-Length: 1905  
Date: Tue, 22 Sep 2026 11:09:13 GMT  
ubuntu@ip-172-31-28-38:~$ sudo systemctl status tomcat10  
● tomcat10.service - Apache Tomcat 10 Web Application Server  
Loaded: loaded (/usr/lib/systemd/system/tomcat10.service; enabled; preset: enabled)  
Active: active (running) since Tue 2026-09-22 11:03:56 UTC; 5min ago  
Invocation: 84365ff827bf40699886237ba0e6f70a  
Docs: https://tomcat.apache.org/tomcat-10.0-doc/index.html  
Main PID: 4238 (java)  
Tasks: 29 (limit: 627)  
Memory: 104.4M (peak: 104.5M)  
CPU: 6.302s  
CGroup: /system.slice/tomcat10.service  
└─4238 /usr/lib/jvm/default-java/bin/java -Djava.util.logging.config.file=/var/lib/tomcat10/conf/logging.properties -Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager -Djava.awt.headless=true  
Sep 22 11:03:57 ip-172-31-28-38 tomcat10[4238]: OpenSSL successfully initialized [OpenSSL 3.5.5 27 Jan 2026]  
Sep 22 11:03:58 ip-172-31-28-38 tomcat10[4238]: Initializing ProtocolHandler ["http-nio-8080"]  
Sep 22 11:03:58 ip-172-31-28-38 tomcat10[4238]: Server initialization in [1637] milliseconds  
Sep 22 11:03:58 ip-172-31-28-38 tomcat10[4238]: Starting service [Catalina]  
Sep 22 11:03:58 ip-172-31-28-38 tomcat10[4238]: Starting Servlet engine: [Apache Tomcat/10.1.55 (Ubuntu)]  
Sep 22 11:03:58 ip-172-31-28-38 tomcat10[4238]: Deploying web application directory [/var/lib/tomcat10/webapps/ROOT]  
Sep 22 11:04:01 ip-172-31-28-38 tomcat10[4238]: At least one JAR was scanned for TLDs yet contained no TLDs. Enable debug logging for this logger for a complete list of JARs that were scanned but no TLDs were  
Sep 22 11:04:01 ip-172-31-28-38 tomcat10[4238]: Deployment of web application directory [/var/lib/tomcat10/webapps/ROOT] has finished in [2,691] ms  
Sep 22 11:04:01 ip-172-31-28-38 tomcat10[4238]: Starting ProtocolHandler ["http-nio-8080"]  
Sep 22 11:04:01 ip-172-31-28-38 tomcat10[4238]: Server startup in [2889] milliseconds  
[lines 1-22/22 (END)]
```





6.Create a Dashboard and monitor the Nginx service to send the alert if Nginx is not running.

Task 6 — Monitor Nginx Service Using CloudWatch Dashboard & SNS Alert

Task: Create a dashboard, monitor the Nginx service, and send an email alert if Nginx is not running.

1. Install and start Nginx

```
sudo apt update
```

```
sudo apt install nginx -y
```

```
sudo systemctl start nginx
```

```
sudo systemctl enable nginx
```

```
sudo systemctl status nginx
```

Verify the web server:

```
curl -I http://localhost
```

Expected:

```
HTTP/1.1 200 OK
```

Server: nginx

2. Verify AWS CLI

```
aws --version
```

The EC2 instance needs permission to publish custom CloudWatch metrics. In our setup, the AWS CLI was already configured/authorized to send the metric.

3. Create the Nginx monitoring script

```
sudo nano /usr/local/bin/nginx-monitor.sh
```

Add:

```
#!/bin/bash
```

```
if systemctl is-active --quiet nginx
then
    STATUS=1
else
    STATUS=0
fi
```

```
aws cloudwatch put-metric-data \
    --namespace "Custom/Nginx" \
    --metric-name "NginxServiceStatus" \
    --value $STATUS \
    --unit Count \
    --region us-east-1
```

```
echo "NginxServiceStatus = $STATUS"
```

Save and exit.

Give execute permission:

```
sudo chmod +x /usr/local/bin/nginx-monitor.sh
```

Test:

```
sudo /usr/local/bin/nginx-monitor.sh
```

Expected while Nginx is running:

```
NginxServiceStatus = 1
```

Here:

1 = Nginx running

0 = Nginx stopped

4. Schedule monitoring using Cron

Open root crontab:

```
sudo crontab -e
```

Add:

```
* * * * * /usr/local/bin/nginx-monitor.sh >> /var/log/nginx-monitor.log 2>&1
```

This checks Nginx **every 1 minute**.

Verify:

```
sudo crontab -l
```

Check the log after a minute:

```
sudo tail -5 /var/log/nginx-monitor.log
```

Expected:

```
NginxServiceStatus = 1
```

```
NginxServiceStatus = 1
```

5. Verify the custom metric in CloudWatch

Go to:

AWS Console → CloudWatch → Metrics → Classic metrics → All

Select:

Custom/Nginx → Metrics with no dimensions → NginxServiceStatus

Select the checkbox.

You should see the metric reporting approximately:

```
NginxServiceStatus = 1
```

6. Create the CloudWatch dashboard

From the metric, choose:

Actions → Add to dashboard

Create:

Nginx-Monitoring-Dashboard

Select:

Widget type: Number

Metric: NginxServiceStatus

Click **Create/Add to dashboard → Save dashboard**.

Your dashboard will show:

NginxServiceStatus

1

7. Create Nginx Down Alarm

Go to:

CloudWatch → Alarms → Create alarm → Select metric

Select:

Custom/Nginx → Metrics with no dimensions → NginxServiceStatus

Configure:

Statistic: Minimum

Period: 1 minute

Threshold type: Static

Condition: Lower

Threshold: 1

The logic is:

1 → Nginx running → OK

0 → $0 < 1$ → ALARM

8. Configure SNS email notification

Under **Configure actions**:

Alarm state trigger: In alarm

SNS topic: Select existing SNS topic

Select your existing SNS topic/email subscription.

Then continue.

9. Give the alarm a name

Use:

Alarm name:

Nginx-Service-Down-Alarm

Description:

This alarm sends an SNS email notification when the Nginx service is stopped or not running.

Then:

Next → Preview → Create alarm

10. Test the alarm

Stop Nginx intentionally:

```
sudo systemctl stop nginx
```

Verify:

```
sudo systemctl status nginx
```

Then wait for the cron job and check:

```
sudo tail -5 /var/log/nginx-monitor.log
```

You should see:

```
NginxServiceStatus = 0
```

CloudWatch now receives:

```
0 < threshold 1
```

Therefore:

OK
↓
ALARM
↓
SNS
↓
Email 

And your screenshot confirms you received:

ALARM: "Nginx-Service-Down-Alarm"

with:

State Change: OK → ALARM
Datapoint: 0.0
Threshold: 1.0

11. Start Nginx again

After testing, don't leave the service stopped:

```
sudo systemctl start nginx  
sudo systemctl status nginx
```

Wait about one minute and verify:

```
sudo tail -5 /var/log/nginx-monitor.log
```

It should return to:

```
NginxServiceStatus = 1
```

CloudWatch will subsequently return the alarm to **OK**.

Final flow to explain in interview/session

"I created a Bash script that checks the Nginx service status. If Nginx is running, the script sends value 1 to a custom CloudWatch metric. If Nginx is stopped, it sends value 0. A cron job executes the script every minute. I created a CloudWatch dashboard to visualize the service status and configured a CloudWatch alarm for values below 1. When Nginx stops, the metric becomes 0, the alarm enters ALARM state, and SNS sends an email notification."

Architecture:

Nginx Service



Bash Monitoring Script



Cron - Every 1 Minute



CloudWatch Custom Metric
(Custom/Nginx)



NginxServiceStatus

1 = Running

0 = Stopped



CloudWatch Dashboard



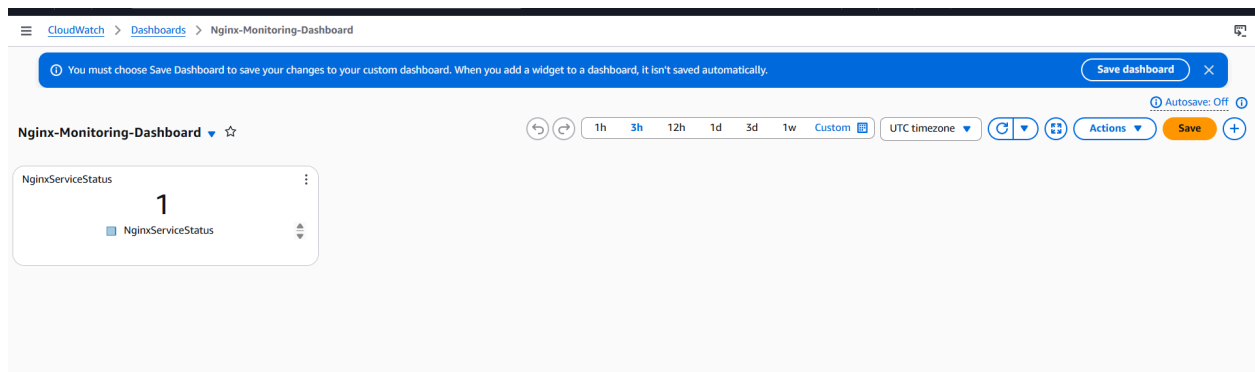
CloudWatch Alarm (< 1)



SNS Topic



Email Alert 



CloudWatch > Alarms

Successfully created alarm Tomcat-Service-Down-Alarm. [View alarm](#)

Alarms (3)

Filter alarms Quick filters

☐	Name	State	Actions	Actions muted	Last state update (UTC)	Conditions
☐	Nginx-Service-Down-Alarm	OK	Actions enabled	-	2026-09-22 12:28:56	NginxServiceStatus < 1 for 1 datapoint within 1 minute
☐	Tomcat-Service-Down-Alarm	OK	Actions enabled	-	2026-09-22 11:54:11	TomcatServiceStatus < 1 for 1 datapoint within 1 minute
☐	EC2-CPU-Utilization-Above-70	OK	Actions enabled	-	2026-09-22 11:03:59	CPUUtilization > 70 for 1 datapoint within 1 minute

CloudWatch > Alarms

Successfully created alarm Nginx-Service-Down-Alarm. [View alarm](#)

Alarms (3)

Filter alarms Quick filters

☐	Name	State	Actions	Actions muted	Last state update (UTC)	Conditions
☐	Nginx-Service-Down-Alarm	In alarm	Actions enabled	-	2026-09-22 12:25:56	NginxServiceStatus < 1 for 1 datapoint within 1 minute
☐	Tomcat-Service-Down-Alarm	OK	Actions enabled	-	2026-09-22 11:54:11	TomcatServiceStatus < 1 for 1 datapoint within 1 minute
☐	EC2-CPU-Utilization-Above-70	OK	Actions enabled	-	2026-09-22 11:03:59	CPUUtilization > 70 for 1 datapoint within 1 minute

Gmail

Compose

Inbox 1

Starred

Snoozed

Sent

Drafts 17

Purchases 3

More

Labels

[imap]/Trash

ALARM: "Nginx-Service-Down-Alarm" in US East (N. Virginia)

AWS Notifications <no-reply@sns.amazonaws.com> to me 5:55 PM (0 minutes ago)

You are receiving this email because your Amazon CloudWatch Alarm "Nginx-Service-Down-Alarm" in the US East (N. Virginia) region has entered the ALARM state, because "Threshold Crossed: 1 out of the last 1 datapoints [0.0 (22/09/26 12:24:00)] was less than the threshold (1.0) (minimum 1 datapoint for OK -> ALARM transition)." at "Tuesday 22 September, 2026 12:25:56 UTC".

View this alarm in the AWS Management Console:
<https://us-east-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-1#alarmsV2:alarm/Nginx-Service-Down-Alarm>

Alarm Details:

- Name: Nginx-Service-Down-Alarm
- Description: This alarm sends an SNS email notification when the Nginx service is stopped or not running.
- State Change: OK -> ALARM
- Reason for State Change: Threshold Crossed: 1 out of the last 1 datapoints [0.0 (22/09/26 12:24:00)] was less than the threshold (1.0) (minimum 1 datapoint for OK -> ALARM transition).
- Timestamp: Tuesday 22 September, 2026 12:25:56 UTC
- AWS Account: 986881030460
- Alarm Arn: arn:aws:cloudwatch:us-east-1:986881030460:alarm:Nginx-Service-Down-Alarm

Threshold:

- The alarm is in the ALARM state when the metric is LessThanThreshold 1.0 for at least 1 of the last 1 period(s) of 60 seconds.

Reply Forward

Task 6: COMPLETED ✓